CSC-696D, Takanori Fujiwara
Final project proposal. **Novel visualization techniques for git data**.

0.1 Problem description

In this project we will create several novel visual analytic systems for git data. The tools we develop will help developers quickly gain an understanding of git repositories. In particular, we seek to give users an understanding of how a repo changes over time. We will focus on a few key questions.

1. Who are the main contributors to a repository
2. At what were the development priorities for a repository at any given time.
3. Can we identify a time when AI assisted development because popular on a given repository.
4. What is the best visualization for giving developers an understanding of the evolution project structure over time.

0.2 Problem description and motivation

Developers often work with large amounts of code and are forced to quickly learn the custom's and conventions of many repositories, this process can be very time-consuming. By giving developers tools for quickly visualizing the history of a repository, we can immediately give them an understanding of conventions, key contributors, and etiquette.

Many visualizations have been developed over the years to work with git data, in this project we want to develop a handful of new visualizations by first studying visualizations that other have developed and refining them to improve there visual clarity, augmenting them with additional data or even combining two visualization techniques into one.

Currently we have planned 3 visualizations but may add more or modify some of them over the course of the project.

1. A 'stabilized' Sediment Graph based on the work Erik Bernhardsson. Bernhardsson proposes a unique way to underhand the age of the contents of a repo, we seek to improve his designs by evenly spacing out various layers of the graph (instead of stacking them), to reduce visual distortions from oscillating streams of data.

2. A commit graph and word cloud style view that allow users to select many commits via a time window selection and see a word cloud of the commit messages. This will allow users to quickly see the development focuses of a repo at a particular time window. This is inspired by a view from the paper *Visual Analytics for Understanding Software Development History Through Git Metadata Analysis*. We seek to enhance there view by adding a time window selection allowing these clouds to be generated for any section of the commit history. Another enhancement we could make is creating an animation going through the commit history with a sliding window to see how the word cloud changes over time.

3. A grouping based visualization that clusters files or directories that are often modified together, this will help users understand which parts of the codebase are most tightly coupled. We seek to animate this view to give users an understanding of how that changes overtime.

0.3 Related research papers

- Githru Visual Analytics for Understanding Software Development History Through Git Metadata Analysis — This work shows an all-in-one visual analytic tool for examining git history and provides a lot of great background on git. Many techniques here may be useful in particular the paper provides a lot of detail on the technical aspects of creating visualization's with git meta-data. We also took inspiration from this paper for our second proposed visualization as discussed above.

- Git-Truck: Hierarchy-Oriented Visualization of Git Repository Evolution — Git-Truck is a hierarchical file-organized-oriented visualization tool that represents git repositories as different treemaps. Each treemap has its nodes sized by the number of lines of code in the repository, and the color of its nodes can represent a variety of interested factors such as number of commits, top contributors, or even the last change day. This is showing the current structure of a repository, rather than the temporal focus we have on in this project. Mainly, we emphasize the importance of seeing the historical development of features in a repository.

- ChangePrism:Visualizing the Essence of Code Changes This paper gives another approach to visualizing code changes that relies on semantic Analysis. Though this papers focus is more on visualizing individual commits and discreet changes rather than the type of broad analysis of larger repositories we hope to do, but gives some good background on existing visual analytic techniques.

0.4 Planned technical approach

Our visualizations will use the '.git' files for the repos we are studying as a standardized source of data. We plan to implement our visualizations in JavaScript, we may pull in dependencies for creating charts as needed. Currently, we do not have our exact technical stack but our project will run on Node, and use NPM for dependency management React is an option but we're also investigating if a simpler UI framework would meet our needs. AI use is planned to speed up implementation, models used will probably be Claude Haiku/Sonnet/Opus 4.x.

0.5 Selected dataset(s)

We will focus on three categories of repos in this project.

- Large repositories with many contributes like the Linux kernal
- Large repositories with few contributes like the emacs
- Small repositories with few contributes like mods and web applications created by small teams or individuals. For fun, we will even feed the git repository for this project into our system.

0.6 Expected results and evaluation methodology

In this project, we seek to create a web application to demonstrate and explain our visual analytic systems. It will feature the visualizations we've created, a short text description of the visualizations—explaining works inspiring the visualization, the types of analytical questions the visualization seeks to answer and the strengths and weakness of our approach. The app will also feature a selector that will allow you to view any git repository with any of the aforementioned visualization techniques. We will the gather feedback from professional software engineers on the usefulness of our visualization and applicability of our tools to there current development workflow.